CPS-RTC
CYBER-PHYSICAL SYSTEMS
RESEARCH AND TECHNOLOGY CENTER

Extracción de datos del HP 8753D

Interfaz GPIB y software

Carlos Valle (cgvalle@uc.cl)
Juan Ignacio Lorca (juan.lorca@uc.cl)

15 de octubre de 2026

1. Interfaz necesaria: GPIB

El analizador de redes **HP 8753D** no dispone de un puerto de red para comunicación con un computador. Los puertos RS-232 y paralelo ubicados en el panel posterior están destinados principalmente a la impresión o generación de copias de la pantalla y no permiten realizar una adquisición por software de los datos.

La interfaz de comunicación adecuada para controlar el instrumento y extraer sus datos numéricos es el bus **GPIB (HP-IB, IEEE-488)**.



Figura 1: VNA HP8753D. Se observan conexiones GPIB, puerto paralelo y serial.

Por lo tanto, es necesario utilizar un adaptador GPIB conectado al computador. La dirección GPIB predeterminada del HP 8753D es **16**.

2. Adaptador GPIB recomendado

Una alternativa disponible es el **XyphroLabs UsbGpib V2**[1], que permite conectar directamente el instrumento a un computador mediante USB.

Sus principales características son:

- Precio aproximado: 59 USD.
- Conexión al computador mediante USB tipo C.
- Conexión al instrumento mediante conector IEEE-488 de 24 pines.
- Implementación basada en **USBTMC**, con soporte de IEEE-488.2.
- Soporte para funciones como lectura del status byte, trigger y service requests.
- No requiere drivers propietarios, ya que utiliza una clase USB estándar.
- Compatible con los principales entornos VISA, incluyendo NI-VISA, Keysight VISA y PyVISA.
- Compatible con **Windows, macOS, Linux y FreeBSD**.

La compatibilidad con USBTMC es especialmente conveniente para trabajar desde **macOS**, ya que permite utilizar el adaptador sin depender de un driver propietario específico del fabricante. Cabe distinguir el *driver* del adaptador, que no es necesario, de la *biblioteca VISA* con que se comunica el software: en macOS y Linux se usa `pyvisa-py`, escrita en Python, mientras que en Windows se requiere instalar NI-VISA o Keysight IO Libraries (Sección 4.2).

La Figura 2 muestra el adaptador instalado en el analizador. Se conecta directamente al puerto GPIB del panel posterior, sin cable IEEE-488 intermedio, y se une al computador mediante un cable USB tipo C. El LED del adaptador indica que está energizado.

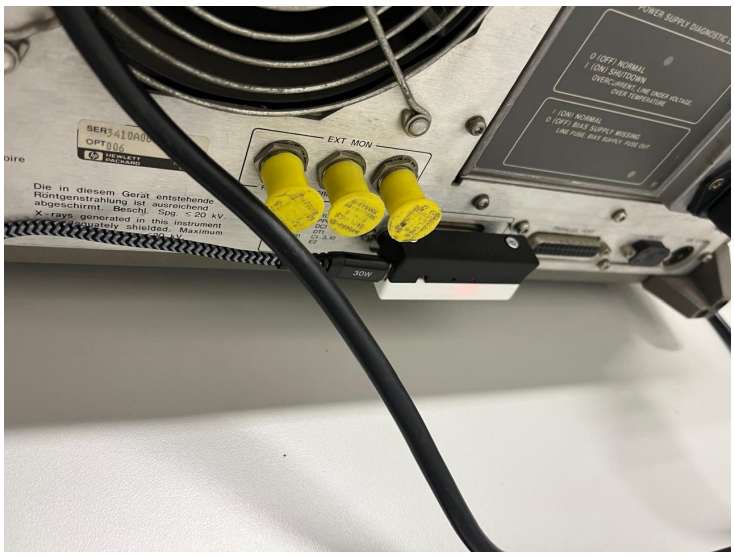


Figura 2: Adaptador XyphroLabs UsbGpib conectado al puerto GPIB del panel posterior del HP 8753D.

El adaptador identifica al instrumento conectado y se presenta ante el computador con un nombre derivado de su respuesta a `*IDN?`, por ejemplo `HEWLETT_PACKARD_8753D_0_6.14`. Por esto, solo aparece como dispositivo USB cuando el analizador está encendido y conectado.

[1] https://binho.io/test-measurement/products/usbgpib-v2/

Precaución: tras un error de *timeout* el adaptador puede desconectarse del bus USB o dejar de responder. En ese caso basta con desconectarlo y volver a conectarlo.

3. Configuración del instrumento

El analizador no requiere habilitar el puerto GPIB (HP-IB), que siempre está activo. Hay que revisar dos ajustes en el panel frontal, ambos accesibles desde la tecla [LOCAL]:

- **Modo de bus:** pulsar [LOCAL] → TALKER/LISTENER. En este modo el analizador acepta comandos de un computador externo. No se debe usar SYSTEM CONTROLLER con un computador conectado. Al apagar y encender el equipo vuelve automáticamente a TALKER/LISTENER; un PRESET no cambia el modo.
- **Dirección GPIB:** pulsar [LOCAL] → SET ADDRESSES → ADDRESS: 8753 y verificar que sea 16, el valor de fábrica. La pantalla ocupa además la dirección 17. La dirección se guarda en memoria no volátil, por lo que no cambia con PRESET ni al apagar el equipo. Si es otra, escribir 16 [x1]. Ningún otro instrumento del bus debe usar las direcciones 16 o 17.

Al 14 de octubre de 2026, el equipo se encuentra configurado con los valores anteriores, por lo que no es necesario modificar sus parámetros de comunicación.

4. Instalación del software

El procedimiento consiste en instalar Python y las dependencias del sistema, crear un entorno virtual, instalar las bibliotecas requeridas y verificar la detección del adaptador. El adaptador XyphroLabs UsbGpib se identifica mediante el identificador de fabricante (VID) `03eb` y el identificador de producto (PID) `2065`.

Los archivos `requirements.txt`, `get_trace.py` y `hp8753d_connect.py` se encuentran disponibles en https://github.com/cgvalle/HP8753D.

4.1. Dependencias de Python

Los programas requieren las bibliotecas de Python indicadas en la Tabla 1 y, en macOS y Linux, la biblioteca de sistema `libusb`.

Paquete	Versión	Función
`pyvisa`	≥ 1.14	Interfaz VISA de alto nivel
`pyvisa-py`	$= 0.7.2$	Backend VISA en Python para USBTMC
`pyusb`	≥ 1.2	Acceso a dispositivos USB mediante libusb
`matplotlib`	≥ 3.5	Generación de gráficos mediante `-plot`

Cuadro 1: Dependencias de Python.

La versión de `pyvisa-py` se fija en 0.7.2 debido a que las versiones de la serie 0.8.x presentan errores de *timeout* al recibir respuestas que el adaptador transmite en varios paquetes USB, como ocurre durante la lectura de una traza completa.

El archivo `requirements.txt` incluido en el repositorio contiene:

```
# get_trace.py: HP 8753D over GPIB via xyphro UsbGpib adapter
# System dependency: libusb (macOS: brew install libusb, Debian/Ubuntu:
    apt install libusb-1.0-0)
pyvisa>=1.14
```

```
# 0.8.x times out on multi-packet replies from the xyphro adapter (trace
    reads)
pyvisa-py==0.7.2
pyusb>=1.2
# only needed for --plot
matplotlib>=3.5
```

En las subsecciones siguientes se instalan con `pip install -r requirements.txt`, desde el directorio del proyecto y con el entorno virtual activado.

4.2. Windows

1. Instalar **Python** desde https://www.python.org. Durante la instalación, habilitar la opción *Add python.exe to PATH*.

2. Instalar **NI-VISA** desde https://www.ni.com/visa o **Keysight IO Libraries Suite**. Ambos incluyen la biblioteca VISA y el soporte USBTMC.

3. Crear y activar un entorno virtual desde PowerShell:

   ```
   py -m venv venv
   venv\Scripts\activate
   ```

4. Instalar las dependencias de Python:

   ```
   pip install -r requirements.txt
   ```

5. Verificar que el adaptador sea detectado:

   ```
   python -c "import pyvisa; print(pyvisa.ResourceManager().
       list_resources())"
   ```

   En este caso se utiliza la implementación VISA instalada en el sistema, por lo que no se especifica el backend `@py`.

Si la instalación es correcta, debería aparecer un recurso similar al siguiente:

```
USB0::0x03EB::0x2065::HEWLETT_PACKARD_8753D_0_6.14::INSTR
```

4.3. macOS

1. Instalar Python y la biblioteca de sistema `libusb` mediante Homebrew:

   ```
   brew install python libusb
   ```

2. Crear y activar un entorno virtual:

   ```
   python3 -m venv venv
   source venv/bin/activate
   ```

3. Instalar las dependencias de Python:

   ```
   pip install -r requirements.txt
   ```

4. Verificar que el adaptador sea detectado:

```
python -c "import pyvisa; print(pyvisa.ResourceManager('@py').
    list_resources())"
```

Si la instalación es correcta, debería aparecer un recurso similar al siguiente:

```
USB0::1003::8293::HEWLETT_PACKARD_8753D_0_6.14::0::INSTR
```

4.4. Linux

1. Instalar Python, el soporte para entornos virtuales y la biblioteca de sistema `libusb`. En Debian, Ubuntu y Raspberry Pi OS, ejecutar:

   ```
   sudo apt install python3 python3-venv libusb-1.0-0
   ```

2. Crear una regla `udev` para permitir el acceso al adaptador sin privilegios de superusuario:

   ```
   echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="03eb", ATTR{idProduct}=="
       2065", MODE="0666"' | sudo tee /etc/udev/rules.d/99-usbgpib.
       rules
   sudo udevadm control --reload-rules
   sudo udevadm trigger
   ```

   Una vez creada la regla, desconectar y volver a conectar el adaptador para aplicar los cambios.

3. Crear y activar un entorno virtual:

   ```
   python3 -m venv venv
   source venv/bin/activate
   ```

4. Instalar las dependencias de Python:

   ```
   pip install -r requirements.txt
   ```

5. Verificar que el adaptador sea detectado:

   ```
   python -c "import pyvisa; print(pyvisa.ResourceManager('@py').
       list_resources())"
   ```

Si la instalación es correcta, debería aparecer un recurso con el mismo formato que en macOS, ya que ambos sistemas utilizan `pyvisa-py`:

```
USB0::1003::8293::HEWLETT_PACKARD_8753D_0_6.14::0::INSTR
```

Si el adaptador no aparece y existe el dispositivo `/dev/usbtmc0`, el driver USBTMC del kernel lo está ocupando. Se puede liberar con `sudo rmmod usbtmc`.

4.5. Prueba de conexión

Con el instrumento en modo **TALKER/LISTENER** y dirección GPIB **16**, ejecutar:

```
python hp8753d_connect.py
```

Si la comunicación se establece correctamente, se debería obtener una respuesta similar a la siguiente:

```
IDN? -> HEWLETT PACKARD,8753D,0,6.14
```

5. Software de adquisición

5.1. Programa propio en Python: `get_trace.py`

La opción recomendada para extraer datos del HP 8753D es el programa `get_trace.py`, disponible en https://github.com/cgvalle/HP8753D. Está escrito en Python, utiliza **PyVISA** y funciona en macOS, Linux y Windows con el adaptador XyphroLabs UsbGpib. Permite automatizar la adquisición de trazas y guardar los datos en formatos estándar sin intervención manual en el instrumento.

Funcionamiento

El programa sigue estos pasos para cada canal solicitado:

1. **Conexión.** Abre el instrumento mediante VISA. En macOS y Linux usa el backend `pyvisa-py` con libusb; en Windows usa NI-VISA. El recurso por defecto es:

   ```
   USB0::1003::8293::HEWLETT_PACKARD_8753D_0_6.14::0::INSTR
   ```

   Como este nombre incluye el modelo y la versión de firmware del instrumento, cambia si se conecta otro analizador. En ese caso, o si el recurso listado en la verificación de la instalación es distinto, se debe indicar con la opción `-resource`.

2. **Configuración opcional.** Si se indica, selecciona el canal (`CHAN1`/`CHAN2`), cambia el formato de visualización (por ejemplo `SMIC` para carta de Smith) y realiza un barrido único con `OPC?;SING;`. Este comando responde solo cuando el barrido ha terminado y deja el analizador en modo *HOLD*.

3. **Lectura del estado.** Consulta la frecuencia inicial y final (`STAR?`, `STOP?`), el número de puntos (`POIN?`) y si el barrido es lineal (`LINFREQ?`). Además identifica el parámetro medido (`S11`, `S21`, etc.) y el formato activo (`LOGM`, `PHAS`, `SMIC`, etc.) consultando cada opción hasta encontrar la que responde `1`.

4. **Transferencia de la traza.** Solicita los datos en formato ASCII (`FORM4`):

   - `OUTPFORM`: la traza tal como se muestra en pantalla (por ejemplo, en dB para `LOGM`).
   - `OUTPDATA`: los datos corregidos por errores como pares real/imaginario, independientes del formato de pantalla. Se usa para generar el archivo Touchstone.

5. **Eje de frecuencias.** El instrumento no entrega las frecuencias, por lo que se reconstruyen a partir del estado leído. En un barrido lineal los puntos se distribuyen uniformemente entre `STAR` y `STOP`; en caso contrario se asume un barrido logarítmico.

6. **Guardado.** Los archivos se guardan en la carpeta `vna_traces/`, con nombres del tipo `trace_<fecha>_ch<canal>_<parámetro>_<formato>`.

Archivos generados

Archivo	Cuándo	Contenido
`.csv`	Siempre	Frecuencia y los dos valores de la traza formateada
`.csv` (Smith)	Formato `SMIC`	Coeficiente de reflexión Γ e impedancia $R + jX$
`_smith.svg`	Formato `SMIC`	Carta de Smith en tamaño carta, con marcadores y tabla
`.s1p`	`-s1p`	Datos corregidos en formato Touchstone (RI, 50 Ω)
`.png`	`-plot`	Gráfico de la traza (requiere matplotlib)

Cuadro 2: Archivos generados por `get_trace.py`.

En formato `SMIC`, la impedancia se calcula a partir del coeficiente de reflexión como

$$Z = Z_0 \frac{1+\Gamma}{1-\Gamma}, \qquad Z_0 = 50\ \Omega.$$

La carta de Smith incluye cuatro marcadores: inicio (1), centro (2), fin (3) y el punto de mejor adaptación (M, menor $|\Gamma|$). Para cada uno se tabulan la frecuencia, $|\Gamma|$, el ángulo, R, X y la ROE (SWR).

Uso

```
python3 get_trace.py                              # canal activo, traza
    formateada -> CSV
python3 get_trace.py --chan 1 2                   # ambos canales
python3 get_trace.py --sweep                      # barrido unico antes de
    leer
python3 get_trace.py --s1p                        # ademas guarda datos
    corregidos (.s1p)
python3 get_trace.py --plot                       # ademas guarda un grafico
     PNG
python3 get_trace.py --chan 1 --format SMIC       # carta de Smith: CSV +
    SVG
```

Opción	Descripción
`-chan 1 2`	Canal(es) a leer; por defecto, el canal activo
`-format FMT`	Cambia el formato antes de leer (`LOGM`, `PHAS`, `SMIC`, ...)
`-sweep`	Realiza un barrido único y deja el equipo en *HOLD*
`-s1p`	Guarda los datos corregidos como Touchstone
`-plot`	Guarda un gráfico PNG
`-outdir DIR`	Carpeta de salida (por defecto `vna_traces/`)
`-resource RES`	Recurso VISA del instrumento

Cuadro 3: Opciones de línea de comandos.

Fragmento principal

El núcleo de la comunicación es la clase `HP8753`. La lectura de una traza se reduce a seleccionar el formato ASCII, enviar el comando de salida y separar los valores en pares:

Listing 1: Lectura de una traza en `get_trace.py`

```
def _read_pairs(self, cmd):
    self.v.write('FORM4;')  # transferencia ASCII
    raw = self.q(cmd)
    vals = [float(s) for s in raw.replace('\n', ',').split(',') if s.
        strip()]
    return list(zip(vals[0::2], vals[1::2]))

def formatted(self):
    return self._read_pairs('OUTPFORM;')   # traza como en pantalla

def corrected(self):
    return self._read_pairs('OUTPDATA;')   # datos corregidos (real,
        imag)
```

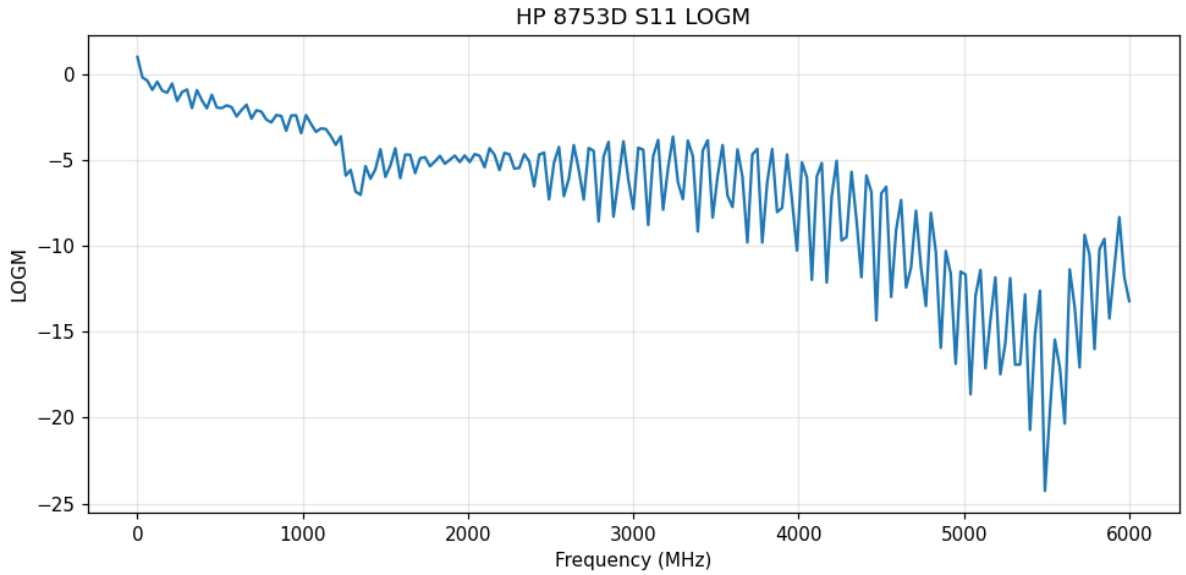


Figura 3: Ejemplo de adquisición de datos mediante `get_trace.py`.

Consideraciones

- Si se usa `-sweep`, el analizador queda en *HOLD*. Para volver al barrido continuo se pulsa **[MENU] → TRIGGER MENU → CONTINUOUS** en el panel frontal, o se envía el comando `CONT;`.
- Ante un error de comunicación persistente (*timeout*), se recomienda desconectar y volver a conectar el adaptador USB.
- Los barridos de tipo lista (*list frequency*) no están soportados: el eje de frecuencias solo se reconstruye para barridos lineales o logarítmicos.

5.2. Alternativa: HP8753 Companion

Existe también una aplicación de código abierto desarrollada específicamente para la familia HP 8753: **HP8753 Companion** (https://github.com/VK2BEA/HP8753-Companion). Está escrita en C con GTK4, se distribuye bajo licencia Apache-2.0 y es compatible con adaptadores USBTMC como el UsbGpib.

Entre sus funciones se encuentran:

- Guardar y restaurar la calibración y la configuración del instrumento.
- Transferir y visualizar trazas en una interfaz gráfica.
- Exportar datos en formato **CSV** y **Touchstone S2P**.
- Guardar trazas como imágenes PNG o PDF y capturar la pantalla en HPGL.
- Importar kits de calibración `.XKT` y enviarlos al analizador.

Su principal ventaja frente a `get_trace.py` es que permite **respaldar y restaurar la calibración** del instrumento. Su limitación es que **solo funciona en Linux**: depende de GTK4, linux-gpib y CUPS, y su documentación cubre Fedora y Raspberry Pi/Debian. No existe versión para Windows ni macOS.